

tidySTL: Exposing Implicit Semantics in STL Evaluation

Sota Sato¹ and Masaki Waga²

¹ National Institute of Advanced Industrial Science and Technology (AIST)

² Kyoto University

Abstract. Robust semantics of Signal Temporal Logic (STL) provides a verdict, optimization objective, and training feedback across monitoring, falsification, and control. Evaluating a finite sampled trace, however, requires decisions that neither the STL formula nor the trace fully specify, such as interpolation and end-of-trace behavior. We call this behavior *implicit semantics*, which can differ among tools and lead to different downstream consequences such as falsification outcomes. We present tidySTL, a diagnostic toolkit that makes such evaluator behavior explicit and exposes their computations through an inspectable DAG. By modeling existing STL evaluators as validated compatibility backends, tidySTL makes their discrepancies transparent and mechanically localizable to responsible formula nodes and operations. Its architecture also serves as an extensible evaluator layer: representative variations require only modular backend changes, while the formula, signal, and public interface remain fixed.

1 Introduction

Signal Temporal Logic (STL) is a standard formalism for specifying temporal properties of real-valued signals [13,9]. Its quantitative *robustness* semantics measures how strongly a behavior satisfies or violates a specification, and the resulting margin underpins runtime monitoring [3], falsification [1], and learning-enabled design [17].

The robustness semantics is defined over an idealized continuous signal, yet a tool usually evaluates it on a finite trace—and a formula and trace do not determine a robustness value on their own. Each tool fixes the rest, silently; we call this *implicit semantics* and organize the choices in Table 1. These choices are not cosmetic: they can flip a tool’s verdict and propagate downstream. Prior cross-tool studies surfaced such disagreements by post-hoc independent checking [8]; our work closes this gap, making the responsible choice explicit and switchable before evaluation.

We present tidySTL,³ a diagnostic toolkit that does exactly this. It places the implicit choices behind a backend boundary, so the same formula and signal can be evaluated under named, swappable backends while the parser, formula representation, and public interface stay fixed (Figure 1). Each backend exposes its

³ Artifact repository: <https://github.com/midoriao/tidystl>.

choices as inspectable intermediate computations; compatibility backends emulate existing tools for cross-tool diagnosis, and the same separation makes the evaluator layer extensible (Table 3).

This paper makes the following contributions.

- We organize the implicit semantic choices in STL robustness evaluation and show that they materially affect acceptance, verdicts, and rankings (§2 and §4.2).
- We present an architecture that represents these choices as explicit, swappable backends and makes their computations inspectable (§3).
- Building on it, we emulate Breach [6], RTAMT [15], and TaLiRo [1] as compatibility backends and *mechanically localize* their discrepancies to the responsible implicit choice and formula node (Table 2 and §4.2).
- We show that the architecture provides an extensible evaluator layer: representative variations require modular backend changes while the formula, signal, and public interface remain fixed (§4.1).

Related Work. In this paper, we focus on the comparison of tidySTL with Breach, RTAMT, and TaLiRo, which employ the quantitative STL semantics of Donzé and Maler [7] or Fainekos and Pappas [9] as their underlying theory. Appendix C lists some other related tools. Extending tidySTL to support other quantitative STL semantics, such as the RoSI semantics [5], is an interesting future direction.

2 Implicit Semantic Choices in STL Evaluation

We take the quantitative STL semantics of Donzé and Maler [7] as a representative theoretical reference. Together with the closely related framework by Fainekos and Pappas [9], it represents the practical setting used by existing STL monitoring and falsification tools [8]; our focus is on evaluator-level variations within this setting. To fix the scope, we recall only the definitions needed to delimit where implicit semantic choices arise in evaluation. For completeness, Appendix A records the standard full inductive definition and further scope distinctions.

Definition 1 (Signal). A signal over a set of variables $\mathbf{Var}$ is a function $\sigma: \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^{\mathbf{Var}}$.

Definition 2 (STL syntax). STL formulas $\varphi \in \mathbf{Fml}$ are given by:

$$\mathbf{Fml} \ni \varphi ::= \top \mid f(\mathbf{x}) \geq 0 \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 U_I \varphi_2 \mid \varphi_1 R_I \varphi_2 \mid \Diamond_I \varphi \mid \Box_I \varphi.$$

Here f is a function symbol, for example $f(x, y) = 4x - y - 3$, and I is a non-singular interval in $\mathbb{R}_{\geq 0}$, for example $[2, 5]$, $(1, 3]$, or $[0, \infty)$.

Informally, the robust semantics $\rho(\varphi, \sigma, t)$ replaces the Boolean verdict with a signed real-value, e.g., $\rho(x - 3 \geq 0, \sigma, t) = \sigma_x(t) - 3$, which the connectives and temporal operators aggregate by min, max, inf, and sup, e.g., $\rho(\Box_{[0,2]}(x - 3 \geq 0), \sigma, t) = \inf_{t' \in [t, t+2]}(\sigma_x(t') - 3)$.

Table 1. Representative *implicit choices* in robustness evaluation [7], each of which can alter the reported value or acceptance class. Appendix A further explores their scope.

Choice	Description
Finite-trace interpretation	Reading samples as a signal, affects robustness
Signal model	Piecewise-linear Piecewise-constant Discrete
Signal beyond horizon . . .	Extend Empty
Robustness at horizon . . .	Exact Copy-prev
Predicate semantics	Atomic-level semantics and supported forms
Distance metric	Signed margin Euclidean (affects robustness)
Nonstandard predicates . . .	E.g., Equality (affects acceptance/robustness)
Logic fragment	Which parts of the full logic are supported, affects acceptance
Unbounded intervals	Unbounded operators like $\Box_{[0,\infty)}$ supported?
Until/release	U_I and R_I supported?
Open intervals	Bounds such as (a, b) and $(a, b]$ supported?
Top/bottom	$\top(+\infty)$ and $\perp(-\infty)$ supported?
Real-valued bounds	Non-integer temporal bounds supported?
Implementation-specific	Others, affects acceptance/robustness
Evaluator conventions . . .	Remaining corner cases specific to implementations

We call an evaluator decision an *implicit semantic choice* if it instantiates robustness evaluation in a way not determined by the formal definition together with the given formula and trace, and can change acceptance or reported robustness. For example, an evaluator often receives only a finite *timed state sequence* $(t_0, \mathbf{v}_0), \dots, (t_n, \mathbf{v}_n)$ of timestamped samples, whereas the reference semantics is defined over an idealized signal. How this finite representation is interpolated into a signal over $\mathbb{R}_{\geq 0}$ is therefore an implicit semantic choice. Table 1 organizes the choices we consider.

Example 1 (Verdict flip from one implicit choice). Consider the *signal beyond horizon* choice of Table 1. For $\varphi = \Diamond_{[2,4]}(x \geq 0)$ evaluated at $t = 1$ on a trace ending at $T = 2$ with $x(T) = 3$, the window $[3, 5]$ lies entirely past the trace. A clamping rule treats the last sample as persisting and reports $+3$ (satisfied), while a pessimistic rule sees an empty window and reports $-\infty$ (violated): same formula, same samples, opposite verdicts.

This is not a knife-edge coincidence of one trace: swept over a family of inputs, the boundary- and terminal-step rules flip verdicts and reverse trace rankings across a wide band of values (Appendix B). The consequences compound when tools are compared. One tool’s implicit choice can contaminate another tool’s verdict: in the ARCH-COMP falsification competition, a tool’s implicit choice of temporal and input bounds invalidated the validation of *other* tools’ correct

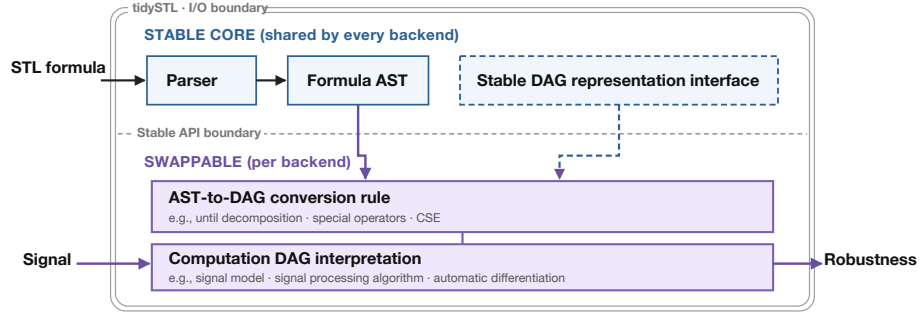


Fig. 1. Architecture of tidySTL: a stable core shared by every backend and a swappable backend layer (lowering and node interpretation) that isolates the implicit choices.

results, turning an implicit choice into a fairness problem for the comparison itself [8].

3 Design and Implementation

This section turns the implicit choices of §2 into inspectable implementation structure. tidySTL exposes robustness evaluation through a small interface, while placing choices that vary across evaluators behind a backend boundary.

3.1 Evaluation Interface

tidySTL has two explicit inputs: formula and signal. A *formula* is a typed syntax tree, parsed from the textual STL front end. A *signal* contains one or more named, batched value sequences over a shared time grid.

An optional third input, the *backend*, encapsulates the implicit choices and the execution strategy. Evaluation takes the form `robustness(formula, signal, backend)` and returns an array of robustness values over time:

```
from tidystl import Signal, parse, robustness
import numpy as np

phi = parse("G[0,5](F[0,2](velocity >= 10))")

times = np.linspace(0, 10, 200)
velocity_batch = np.array([np.sin(times), np.cos(times)]) # two traces
sig = Signal.from_dict(
    times=times,
    values={"velocity": velocity_batch},
)
rho = robustness(phi, sig) # default backend
rho = robustness(phi, sig, backend="breach") # Breach-compatible
```

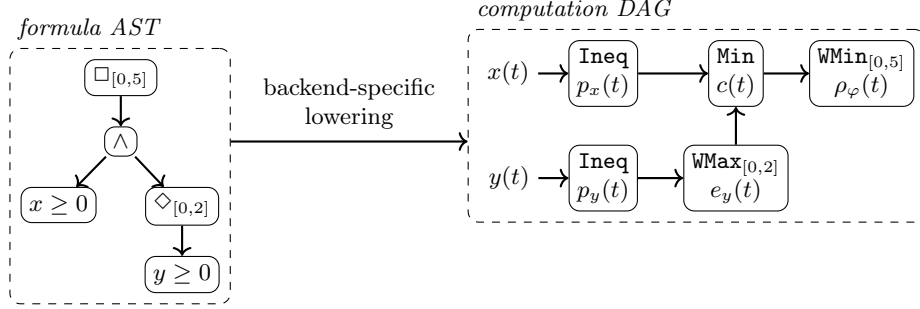


Fig. 2. Lowering of $\varphi \equiv \Box_{[0,5]}((x \geq 0) \wedge \Diamond_{[0,2]}(y \geq 0))$ from AST (left) to computation DAG (right). Each node is a primitive operation: the conjunction reduces pointwise while the temporal operators aggregate over their annotated window. Edges carry intermediate robustness signals that remain inspectable in the evaluation result.

Keeping this interface small serves two purposes. For users, the same formula and signal can be evaluated under different documented backends without changing the surrounding application. For backend authors, new evaluation behavior attaches below a stable interface rather than requiring a new parser, signal representation, or calling convention.

3.2 Architecture

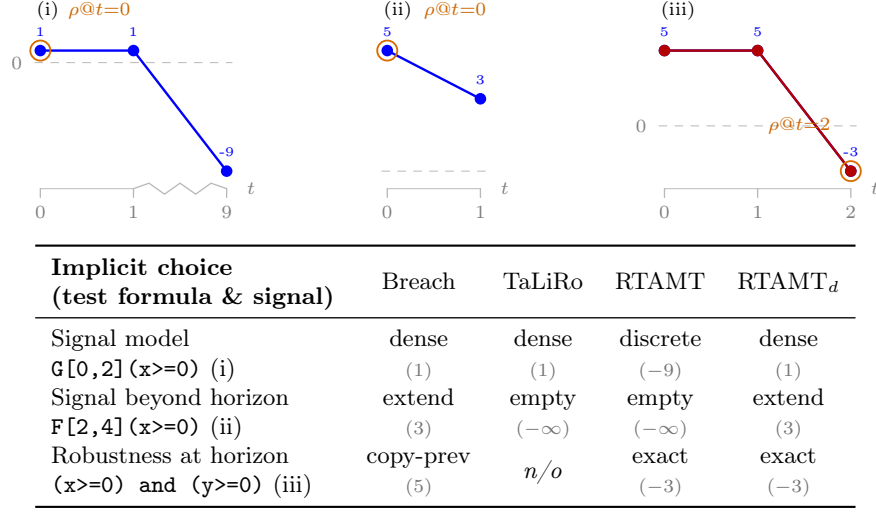
Internally, tidySTL separates a stable core from backend-specific evaluation logic, as shown in Figure 1. The stable core is shared by every backend, and it does not decide how (φ, σ) maps to a robustness value. Those decisions are delegated to the backend layer.

This separation provides *observability*: because two backends build their intermediate outputs on the same shared formula (syntax-tree) representation, those outputs can be aligned at corresponding formula nodes. This makes it possible to identify the minimal divergent node and inspect the signal-processing operations responsible for it. §4.2 demonstrates this diagnostic workflow.

Each backend lowers the formula AST into a backend-specific computation DAG, and then interprets its typed primitive operations as a concrete signal-processing pipeline. This keeps signal-processing factors separate from the syntactic and recursive handling of STL formulas. Figure 2 illustrates such a lowering on an example formula with nested temporal operators.

Extension points. A backend consists of primitive operations, a lowering rule, and an executor. Extensions may introduce new primitive operations during lowering or override the interpretation of existing ones. For programming details, the artifact repository gives the user manual (`docs/usage.md`) and reference implementations. In §4.1, we demonstrate that emulating an existing tool’s behavior requires only backend changes with little effort. A performance sanity check finds the overhead of this structure negligible (Appendix B).

Table 2. Implicit-choice profile across three finite-trace axes (cf. Table 1) for Breach, TaLiRo, and RTAMT (discrete and dense modes). *Top*: the three diagnostic input signals, named next to each row’s test formula; samples are spaced evenly by index (the t -axis gives each sample’s real time) and the orange ring marks the readout sample whose robustness the table reports. *Bottom*: each row varies only the named choice; each cell gives the implicit choice revealed at the readout sample, with the observed value in parentheses. n/o : choice not observable in TaLiRo’s whole-trace scalar output.



Architectural byproducts. The swappable architecture (Figure 1) also supports capabilities outside the central scope of this paper. For example, it can accommodate advanced semantic extensions, such as smooth differentiable robustness [11], as well as performance-optimized backends, all through the same interface. The codebase also contains differentiable and SIMD-optimized backends as instances of this extensibility.

4 Evaluation

We evaluate tidySTL in two steps.⁴ First, we show that realizing an implicit choice in tidySTL is a modular change confined to the backend layer, and that the resulting compatibility backends faithfully reproduce existing tools (§4.1). Second, building on these validated backends, we localize cross-tool discrepancies to the responsible choice and formula node (§4.2).

We fix a single set of reference tools throughout: Breach 1.11.4, RTAMT 0.3.5 (discrete default and dense, RTAMT_d), and TaLiRo (SVN revision 146). The implicit choices of §2 already produce discrepancies on those tools: Table 2

⁴ The artifact repository provides reproduction instructions ([extra/experiments/](#)) for all experiments; Appendix B records supplementary protocol and setup details.

Table 3. Implemented compatibility backends: layer changed and variant-specific LOC (blank/comment lines excluded) as an auditable size proxy. All 103 regression tests pass with every variant installed.

Variant	Layer changed	LOC
Breach-compatible	executor op overrides	85
TaLiRo-compatible	own lowering + executor	167
RTAMT-compatible	own lowering + executor	191

Table 4. Tool-vs-tool divergence localization with no ground truth. Each case lists the two tools compared, the formula size/operator depth, the root robustness each tool reports (ρ , left/right tool), and the localized divergence (the single origin operator the localizer blames). The cases use `until_boundary` $(x \geq 0) U_{[1,2]} (y \geq 0)$ and `window_sampling` $\Box_{[0,3]} ((mode \geq 0) \wedge \Diamond_{[0,1]} ((enable \geq 0) \wedge \Box_{[1,2]} (signal \geq 0))) \wedge (safety_margin \geq 0)$.

Case	tools compared	$ \phi /\text{depth}$	ρ (l/r)	divergence
<code>until_boundary</code>	Breach vs. RTAMT	3 / 1	-5 / +2	verdict flip on $U_{[1,2]}$
<code>window_sampling</code>	Breach vs. TaLiRo	10 / 3	+2 / +5	value gap on $\Box_{[1,2]}$

profiles three finite-trace axes and every one splits the tools into at least two camps, with no tool wrong.

4.1 Implementing Evaluation Variations as Modular Changes

We first measure the implementation cost of realizing an implicit choice in tidySTL. Table 3 shows that the shipped compatibility backends change only backend-side components, as illustrated in Figure 1. Each backend implements primitive operations, lowering rules, and executors below the stable core.

Compatibility validation. Each reproduces its tool’s robustness on the operator-coverage regression suite—per-timestep for the Breach- and RTAMT-compatible backends, and at $t = 0$ only for TaLiRo, which reports just that value—validated case by case against the tool’s committed reference fixtures by the artifact’s reproduction test suite (Appendix B).

4.2 Localizing Cross-Tool Discrepancies

The discrepancies of Table 2 have a mechanical cause that tidySTL can name. Using the validated backends of §4.1, its localizer compares a diverging pair node by node: every backend lowers the formula into one shared syntax tree with inspectable per-node outputs, so a single post-order walk reports the *minimal divergent node*—the lowest node whose backend outputs disagree while all of its children still agree—together with the primitive operation each backend runs there (Table 4).

Consider the `window_sampling` case of Table 4, the conjunction

$$\Box_{[0,3]}((mode \geq 0) \wedge \Diamond_{[0,1]}((enable \geq 0) \wedge \Box_{[1,2]}(signal \geq 0))) \wedge (safety_margin \geq 0).$$

It can be read as a small CPS-style monitoring requirement with a mode signal, an enabling signal, a monitored continuous signal, and a safety margin: during the window $[0, 3]$, the system must remain in the required mode; within the initial window $[0, 1]$, it must become enabled; once enabled, the monitored signal must stay above its threshold throughout the following $[1, 2]$ window; and the run must maintain a nonnegative safety margin. We evaluate it on a non-uniform trace whose grid jumps from $t = 0$ straight to $t = 3$ (samples at $t = 0, 3, 4, \dots, 9$), with $mode = enable = 10$ and $safety_margin = 9$ held constant and $signal = [3, 0, 5, \dots, 5]$. Breach reports root robustness $+2.0$ and TaLiRo $+5.0$: both satisfy the spec, yet the values differ, so no flipped verdict betrays the divergence.

The localizer pins the gap to the buried $\Box_{[1,2]}$, three temporal levels down. Evaluated at the first step, this operator reduces $signal$ over the window $[1, 2]$, which lies entirely inside the $0 \rightarrow 3$ grid gap and so contains no sample. Breach interpolates $signal$ at the window start and returns $+2.0$ (because linear interpolation between the samples at $t = 0$ and $t = 3$ gives $signal(1) = 2$); TaLiRo finds the window empty and returns the always-identity $(+\infty)$, so that branch stops constraining the result. The two local readings propagate unchanged up the three nesting levels: at the root Breach’s interpolated $+2.0$ stays binding, while TaLiRo’s value rises to the next active constraint, $+5.0$. The cause lives entirely in how an off-grid temporal window is reduced; a more readable walkthrough of this example is given in Appendix B.

The verdict can also flip outright. On the bounded-until case $(x \geq 0) U_{[1,2]}(y \geq 0)$, Breach and RTAMT disagree on the root sign (robustness -5.0 vs. $+2.0$; across three tools RTAMT is the lone outlier), and the localizer pins every pairwise disagreement to the single $U_{[1,2]}$ node, i.e. RTAMT’s discrete-time until composition. The analogous contamination in the ARCH-COMP competition, by contrast, was traced down only by manual investigation [8]. In both cases the cause lives in the temporal semantics, which a reader cannot recover from the formula alone.

5 Conclusion and Future Work

tidySTL makes the implicit choices of STL robustness evaluation explicit at the evaluator boundary. By separating formula syntax, signal data, backend-specific lowering, and DAG interpretation, it exposes choices that are otherwise buried in tool internals and mechanically localizes the resulting cross-tool discrepancies to minimal divergent formula nodes and responsible operations.

The implementation supports the view of robustness evaluation as an extensible semantic layer rather than an implementation detail of a particular monitor, falsifier, or learning pipeline. Future work will extend this layer to broader semantics, including online evaluation, and expand its validation suite.

A Scope and Implicit Semantic Choices in Detail

This appendix expands the implicit choices of Table 1 and organizes the sources of cross-tool difference. We first recall the reference robustness semantics, then detail finite-trace interpretation, coverage, and implementation-specific behavior. We then treat advanced semantic extensions, and close by stating what makes all of these choices implicit.

A.1 Standard Robustness

Following Donzé and Maler [7], fix a signal σ in the sense of Definition 1 and a formula $\varphi \in \mathbf{Fml}$ as in Definition 2. The standard quantitative (robustness) semantics $\rho(\varphi, \sigma, t) \in \mathbb{R} \cup \{\pm\infty\}$ is defined inductively:

$$\begin{aligned}
\rho(\top, \sigma, t) &= +\infty \\
\rho(f(\mathbf{x}) \geq 0, \sigma, t) &= f(\sigma_{\mathbf{x}}(t)) \\
\rho(\neg\varphi, \sigma, t) &= -\rho(\varphi, \sigma, t) \\
\rho(\varphi_1 \vee \varphi_2, \sigma, t) &= \max(\rho(\varphi_1, \sigma, t), \rho(\varphi_2, \sigma, t)) \\
\rho(\varphi_1 \wedge \varphi_2, \sigma, t) &= \min(\rho(\varphi_1, \sigma, t), \rho(\varphi_2, \sigma, t)) \\
\rho(\varphi_1 U_I \varphi_2, \sigma, t) &= \sup_{t' \in t+I} \min\left(\rho(\varphi_2, \sigma, t'), \inf_{t'' \in [t, t']} \rho(\varphi_1, \sigma, t'')\right) \\
\rho(\Diamond_I \varphi, \sigma, t) &= \sup_{t' \in t+I} \rho(\varphi, \sigma, t') \\
\rho(\Box_I \varphi, \sigma, t) &= \inf_{t' \in t+I} \rho(\varphi, \sigma, t'),
\end{aligned}$$

where $t+I = \{t+s \mid s \in I\}$ and $f(\sigma_{\mathbf{x}}(t))$ applies f to the values of the variables $\mathbf{x}$ at time t . The release operator is defined by duality, $\varphi_1 R_I \varphi_2 := \neg(\neg\varphi_1 U_I \neg\varphi_2)$, and the constant $\perp := \neg\top$ has $\rho(\perp, \sigma, t) = -\infty$.

Read literally, this definition assumes an idealized signal: continuous, infinitely long, and known at every instant. An evaluator never has that. It has a finite list of samples $\sigma(t_0), \dots, \sigma(t_n)$, and to compute the sup, inf, and min above it must decide what the signal does at the times that fall between, before, and after those samples. Those decisions form the finite-trace interpretation discussed in Appendix A.3.

A.2 Coverage and Implementation Deviations

Several rows of Table 1 are *coverage* choices: adopting a tool implicitly fixes which formulas it can evaluate at all, before any robustness is computed.

Logic fragment. The until operator, with its nested inf, is harder to vectorize than $\Box$ and $\Diamond$, so some tools support only the box/diamond fragment and drop release with it. Definition 2 also admits unbounded intervals such as $[0, \infty)$ and an arbitrary function symbol f ; many tools require a finite upper bound and restrict predicates to threshold comparisons or linear arithmetic. Each omission restricts the accepted language.

Interval endpoints and sampling grid. Definition 2 admits open and half-open intervals such as $(1, 3]$, but many front ends parse only closed ones; where open intervals are accepted, excluding an endpoint changes a discrete inf/sup while leaving a continuous one unchanged. A discrete-time evaluator addresses time by integer index, so it admits only the integer-interval fragment: every bound must be an integer multiple of the sampling period and the trace must be uniform. tidySTL’s RTAMT-compatible backend enforces this as a coverage restriction—on a step-1 trace $\Box_{[1,2]}$ is admitted but $\Box_{[0.5,1.5]}$ is rejected outright, and a non-uniform grid is rejected rather than silently snapped.

Robustness of $\top$ and $\perp$. The reference semantics fixes $\rho(\top) = +\infty$ and $\rho(\perp) = -\infty$. Tools that substitute large finite sentinels deviate from it, and the sentinel leaks into reported values once it is aggregated, scaled, or differentiated. We record this as an implicit choice because adopting the evaluator silently selects the effective behavior.

A.3 Implicit Semantic Choices for Finite Inputs

The most consequential category is the *finite-trace interpretation*: the bundle of implicit choices, summarized in Table 1, by which the finite timed state sequence an evaluator receives is read as the signal σ that the semantics quantifies over. We discuss its choices in turn, then the remaining implicit choices in Table 1.

Signal model. The signal model decides how a sampled trace is read as a function of continuous time. Under a *piecewise-linear* dense-time model (Breach, and tidySTL’s native backend) the value between two samples is their linear interpolant, so a predicate such as $x \geq 0$ can change truth between sample points. A *piecewise-constant* model holds each sample until the next. A *discrete-time* model (RTAMT’s default) drops the notion of between-sample time altogether: the trace is an indexed sequence and temporal operators count indices, not durations. The same formula and the same samples can therefore denote different signals before evaluation even begins.

Window endpoints. Even under a fixed dense signal model, a temporal window $[t + a, t + b]$ whose endpoints fall between samples leaves a choice of which points the inf/sup ranges over. tidySTL’s native backend includes the interpolated values at the exact endpoints alongside the in-window samples, returning the true piecewise-linear extremum over the real interval. The Breach-compatible backend reduces over the in-window *samples only* and never interpolates at an endpoint (pinned to Breach ground truth). Both backends are piecewise-linear, yet they disagree on any off-grid window: for $\Diamond_{[0.3,0.7]}(v \geq 5)$ the native backend reports -0.2 and the Breach-compatible one -1.0 . tidySTL records this among the evaluator-specific behaviors of Table 1, as it reflects a Breach implementation convention rather than a primary finite-trace choice.

The *signal beyond horizon* and *robustness at horizon* rows of Table 1 cover the next two choices: how a window reaching past the trace end is filled, and how the final sample is reported.

Signal beyond horizon (boundary rule). When a window $[t + a, t + b]$ reaches past the end of the trace at T , the sup or inf ranges in part over times where no sample exists. The boundary rule supplies those values, either by *extending* the signal past the horizon or by leaving the past-end region *empty*—the *Extend* and *Empty* entries in Table 1. Under *clamp* (tidySTL, Breach) the last sample is treated as persisting, so an out-of-range query returns $\sigma(T)$. Under a *pessimistic* rule (RTAMT discrete, TaLiRo) the out-of-range part contributes nothing: the window is effectively empty, so a $\diamond$ becomes $-\infty$ (the sup of an empty set) and a $\square$ becomes $+\infty$. RTAMT’s dense mode (RTAMT_d), by contrast, extends the final sample, matching the clamp rule rather than the discrete mode’s pessimistic one. This is the *signal beyond horizon* row of Table 2, illustrated on a concrete trace in Example 1. It is a sign change in the robustness value, not a difference in input acceptance: both backends evaluate the formula, and only the reported value—and hence the verdict—differs. The *signal beyond horizon* rule is therefore a robustness effect, like the signal model.

Robustness at horizon (terminal-step rule). At the final sample t_N a tool may report the *exact* robustness it computed there, or it may *copy* the value from the penultimate sample t_{N-1} —the *Exact* and *Copy-prev* entries in Table 1. Breach does the latter by default; RTAMT does the former. The two agree whenever the final and penultimate robustness coincide, so the choice is silent on most traces. It is not silent when they differ. This is the *robustness at horizon* row of Table 2. For $\varphi = (x \geq 0) \wedge (y \geq 0)$ on trace A with $x = y = [5, 5, -3]$, the penultimate robustness is +5 and the final is −3. The standard rule reports −3; the extension rule copies the penultimate +5. Against a trace B that holds +0.1 throughout, this flips which trace looks better, reversing a ranking that a falsification search would act on.

Robustness of atomic predicates. Even for an accepted predicate, the evaluator must decide how it is scored (the *distance metric* row of Table 1): the robustness of $f(\mathbf{x}) \geq 0$ may be the raw signed margin $f(\mathbf{x})$ or a Euclidean distance to the satisfaction set [9], and the two disagree once a predicate involves several variables.

A.4 Advanced Semantic Extensions

Applications often demand more than Definition 2 provides. The extensions below select a different language or evaluation problem; whether a tool supports one is itself a coverage choice.

Equality predicates. The equality $x = 0$ is absent from Definition 2 yet common in practical specifications. Once a tool accepts it the assigned robustness is an implicit choice (the *nonstandard predicates* row of Table 1): a common reading is $-|x|$, which is never positive, so even a satisfied equality reports non-positive robustness, while Breach uses a fixed-magnitude sign convention ($\pm 10^4$) that carries no metric information.

Other extensions. Past-time operators (once, historically), differentiable robustness that replaces min/max by smooth approximations [12,16], and online evaluation over growing prefixes [5] each change the language or the evaluation problem; tidySTL exposes the smooth case explicitly as a selectable backend.

A.5 Why These Choices Are Implicit

Across the four groups of Table 1—finite-trace interpretation, predicate semantics, logic fragment, and implementation-specific behavior—one property recurs: the choice is fixed not by the STL formula or the sampled trace, but by the act of adopting an evaluator. That is what makes it implicit and invisible at the specification level. The extensions of Appendix A.4 sharpen this boundary rather than crossing it: a user who explicitly selects, say, smooth robustness makes that choice visible, yet the tool’s behavior around the extension—which inputs it admits and how it scores them—stays implicit. “Choice” here is descriptive, not a verdict on a tool’s documentation or design: it names the effective behavior without judging whether a tool fixed it deliberately. The practical consequence is what §4.2 measures: two tools whose combinations differ can reject different inputs, report different robustness, or return opposite verdicts on identical input.

B Experimental Details

This appendix records the protocols and full data behind the findings reported in §4.

Artifact and reproducibility. The public artifact repository contains the implementation, fixtures, and reference-tool adapters. The user manual documents installation, backend semantics, and worked examples; the experiment guide documents how to regenerate every evaluation record, table, and figure.

Diagnostic case set and protocol. The profile of Table 2 uses one diagnostic case per implicit choice, each a small formula and trace constructed so that only that choice can affect the reported robustness. Every case runs on each tool in its native environment—Breach (from a committed MATLAB cache), RTAMT in its discrete and dense modes, and TaLiRo (`dp_taliro`)—and we read the tool’s per-timestep robustness at the marked readout sample and map it to the option that output reveals. The per-option robustness signature is predicted analytically and printed beside the observed value, so each attribution is auditable rather than asserted. TaLiRo’s `dp_taliro` reports only a whole-trace scalar at $t = 0$, so a choice that acts only on a later sample is marked *n/o* (not observable) rather than guessed.

Compatibility validation. Each tidySTL compatibility backend is validated separately against its tool’s committed reference fixtures—real-tool robustness regenerated from Breach, RTAMT, and TaLiRo—by the artifact’s per-tool reproduction test suite at absolute tolerance 10^{-6} on the operator-coverage regression

suite. The check is per-timestep for the Breach- and RTAMT-compatible backends; for TaLiRo it is at $t = 0$ only, the single value `dp_taliro` reports. The backends implement each tool’s *documented* rules, so the check verifies the implementations rather than defining the rules. The one case where a backend departs from its reference tool is itself an implicit choice this paper aims to surface: where real RTAMT silently reinterprets a non-uniform grid as uniformly indexed samples, the RTAMT-compatible backend makes this an *explicit* coverage rejection rather than reproducing the silent reinterpretation (cf. the coverage choices of Appendix A.2).

Localization procedure. Across any pair of validated backends, the localizer exploits that tidySTL backends share one formula representation with inspectable per-node outputs: it walks the shared syntax tree in post-order and reports the *minimal divergent node*—the lowest node whose outputs disagree while all its children agree—together with each backend’s primitive operation at that node. To illustrate the output format, on two minimal single-operator cases (the *signal beyond horizon* and *robustness at horizon* choices) the listing reads:

```
first divergent op at 'F[2,4]': WindowMax[2,4] vs
    DiscreteWindowMax[2,4] (t-index 1) -> boundary rule
first divergent op at 'and': PointwiseMin vs PointwiseMin
    + terminal-step extension (t-index 2) -> terminal-step rule
```

The headline results are the tool-vs-tool localizations of §4.2 (Table 4 and Figure 3), where the same post-order walk pins a cause buried under several nesting levels. Backend pairs whose lowerings differ structurally are compared at the shared syntax-tree level rather than between lowered representations; predicate-internal arithmetic is treated as a leaf, since none of the implicit choices studied here concerns its internal operations.

Trace-family construction. A single constructed trace could sit on a knife edge, so each headline case is widened to a swept family of 201 traces whose final sample varies over $[-5, 5]$, spanning regions of agreement and disagreement; no rule is taken as the reference, and the rates are symmetric between the rules by construction. For the boundary-rule family ($\Diamond_{[2,4]}(x \geq 0)$, window past the trace end), the clamping and pessimistic rules return opposite verdicts at the divergence timestep on exactly those traces whose final sample is non-negative (101 of 201). The terminal-step family is a conjunction violated only at the final sample; the as-is and extend-penultimate rules flip the final verdict on 100 of 201 traces and rank the trace oppositely against a fixed reference trace with an interior violation—the ranking signal a falsification search acts on—on 80 of 201. These rates measure the width of the disagreement region within each controlled family, not a frequency claim about any particular workload.

Variant accounting. LOC in Table 3 counts variant-specific code only: blank, comment-only, and docstring lines are excluded, and shared infrastructure is

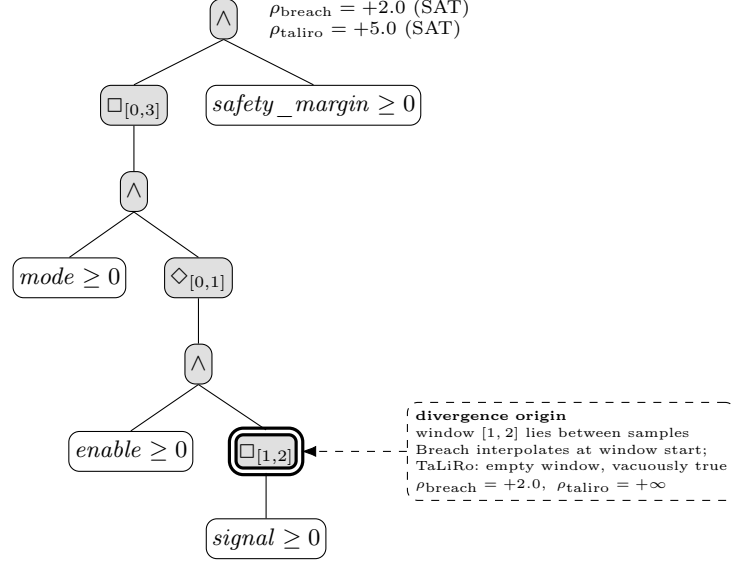


Fig. 3. Tool-vs-tool localization on `div_window_sampling` (BREACH vs. TALiRO, no ground truth; discussed in §4.2). The buried $\square_{[1,2]}$ reduces over a window lying entirely between samples on a non-uniform grid: BREACH interpolates at the window start while TALiRO returns the always-identity. Shaded nodes mark the divergence propagation path up to the single origin $\square_{[1,2]}$ (**WindowMin** vs. **SampleMin**), annotated in the dashed box; both satisfy the spec (+2.0 vs. +5.0).

excluded. The native backend is the baseline and has no row. The full regression suite comprises 103 tests and passes with all variants installed.

Performance sanity check. The benchmark in Figure 4 compares tidySTL backends with real RTAMT and STLGG++ on the same workload. On single traces, the explicit evaluator layer remains within the same order of magnitude as the Python-stack references. On batches, tidySTL’s batch-first signal layout amortizes per-trace cost, making the native backend substantially faster than looping RTAMT and faster than the tensor-batched STLGG++ run shown here. The SIMD executor gives the expected speed ceiling once per-node observability is dropped. This check shows that inspectability does not impose prohibitive overhead; performance is not a primary evaluation claim.

Table 5 tabulates the measurements plotted in Figure 4.

C Comparison with Other Tools

Table 6 lists further tools that monitor the robustness of STL or related logics, classified by their supported nominal semantics (defined in the caption). tidySTL

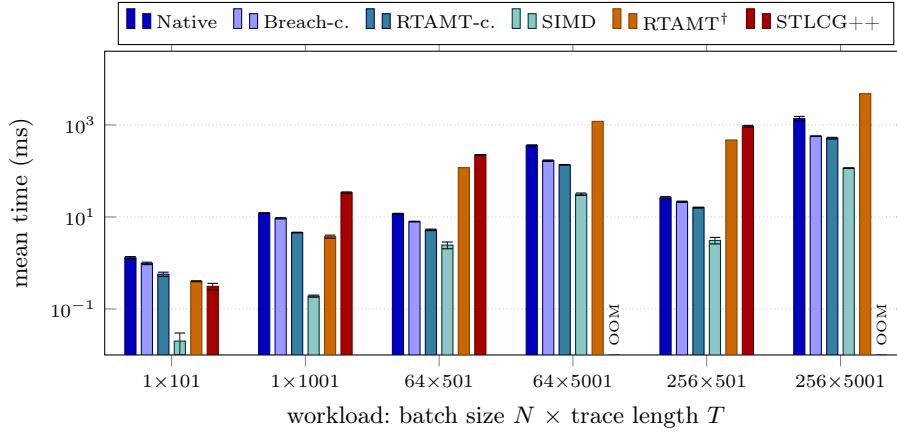


Fig. 4. Mean evaluation time (log scale) on $\Box_{[0,5]}((x > 0) \wedge \Diamond_{[0,2]}(y > 0))$; error bars are one standard deviation. RTAMT†: batch time is $N \times$ per-trace time. Missing STLCG++ bars: OOM.

Table 5. Mean evaluation time (ms) $\pm$ std on the same workload as Figure 4. †: RTAMT is single-trace; batch times are $N \times$ per-trace (linearity checked at $N \in \{1, 8\}$). OOM: window materialization exceeds memory on the 32 GB host.

N	T	Native	Breach-c.	RTAMT-c.	SIMD	RTAMT	STLCG++
1	101	1.30 ± 0.08	0.98 ± 0.06	0.57 ± 0.06	0.02 ± 0.01	0.40 ± 0.01	0.31 ± 0.05
1	1001	12.15 ± 0.19	9.34 ± 0.28	4.58 ± 0.08	0.19 ± 0.01	3.74 ± 0.28	33.71 ± 1.31
64	501	11.75 ± 0.25	7.94 ± 0.15	5.24 ± 0.20	2.44 ± 0.41	$118^\dagger$	224 ± 3
64	5001	358 ± 11	167 ± 6	136 ± 2	31.20 ± 1.89	$1195^\dagger$	OOM
256	501	26.19 ± 1.54	21.42 ± 0.65	15.97 ± 0.37	3.09 ± 0.50	$471^\dagger$	934 ± 44
256	5001	1377 ± 155	569 ± 9	515 ± 23	115 ± 2	$4781^\dagger$	OOM

currently supports the classical and smooth semantics; extending it with compatibility backends for further classical-semantics tools, and for the RoSI-style [5] and causation [21] semantics, is future work that the modular structure of §4.1 should make straightforward.

References

1. Annpureddy, Y., Liu, C., Fainekos, G.E., Sankaranarayanan, S.: S-TaLiRo: A tool for temporal logic falsification for hybrid systems. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, vol. 6605, pp. 254–257. Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_21
2. Bartocci, E., Bortolussi, L., Loret, M., Nenzi, L.: Monitoring mobile and spatially distributed cyber-physical systems. In: Talpin, J., Derler, P., Schneider, K. (eds.) Proceedings of the 15th ACM-IEEE International Conference on Formal Methods and Models for System Design, MEMOCODE 2017, Vienna, Austria, September

Table 6. Tools supporting STL or related robustness monitoring. “classical” semantics: [7] or [9]; “RoSI-style”: [5].

Name	Supported Logic	Supported Nominal Semantics
Breach [6]	Future-time STL	classical
TaLiRo [1]	Future-time STL	classical
RTAMT [20]	Future- and past-time STL	classical
STLCG++ [11]	Future-time STL	classical and smooth [12,16]
SignalTemporalLogic.jl ⁵	Future-time STL	classical and smooth [12,16]
py-signal-temporal-logic [19]	Future-time STL	classical
mstlo [18]	Future-time STL	classical and RoSI-style
Argus ⁶	Future-time STL	classical
STLRom ⁷	Future-time STL	RoSI-style
CauMon [22]	Future-time STL	causation semantics [21]
MoonLight [14]	STREL [2]	classical
TempLogic ⁸	Future-time STL, TSSL [4], SpaTeL [10]	classical

29 - October 02, 2017. pp. 146–155. ACM (2017). <https://doi.org/10.1145/3127041.3127050>, <https://doi.org/10.1145/3127041.3127050>

3. Bartocci, E., Deshmukh, J., Donzé, A., Fainekos, G., Maler, O., Ničković, D., Sankaranarayanan, S.: Specification-based monitoring of cyber-physical systems: A survey on theory, tools and applications. In: *Lectures on Runtime Verification: Introductory and Advanced Topics*, Lecture Notes in Computer Science, vol. 10457, pp. 135–175. Springer (2018). https://doi.org/10.1007/978-3-319-75632-5_5
4. Bartocci, E., Gol, E.A., Haghighi, I., Belta, C.: A formal methods approach to pattern recognition and synthesis in reaction diffusion networks. *IEEE Trans. Control. Netw. Syst.* **5**(1), 308–320 (2018). <https://doi.org/10.1109/TCNS.2016.2609138>, <https://doi.org/10.1109/TCNS.2016.2609138>
5. Deshmukh, J.V., Donzé, A., Ghosh, S., Jin, X., Juniwal, G., Seshia, S.A.: Robust online monitoring of signal temporal logic. *Formal Methods in System Design* **51**(1), 5–30 (2017). <https://doi.org/10.1007/s10703-017-0286-7>
6. Donzé, A.: Breach, a toolbox for verification and parameter synthesis of hybrid systems. In: *Computer Aided Verification (CAV)*. Lecture Notes in Computer Science, vol. 6174, pp. 167–170. Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_17
7. Donzé, A., Maler, O.: Robust satisfaction of temporal logic over real-valued signals. In: *Formal Modeling and Analysis of Timed Systems (FORMATS)*. Lecture Notes in Computer Science, vol. 6246, pp. 92–106. Springer (2010). https://doi.org/10.1007/978-3-642-15297-9_9
8. Ernst, G., Arcaini, P., Bennani, I., Chandratre, A., Donzé, A., Fainekos, G., Frehse, G., Gaaloul, K., Inoue, J., Khandait, T., Mathesen, L., Menghi, C., Pedrielli, G., Pouzet, M., Waga, M., Yaghoubi, S., Yamagata, Y., Zhang, Z.: ARCH-COMP 2021 category report: Falsification with validation of results. In: *8th International Workshop on Applied Verification of Continuous and Hybrid Systems (ARCH21)*. EPiC Series in Computing, vol. 80, pp. 133–152. EasyChair (2021). <https://doi.org/10.29007/xw11>

9. Fainekos, G.E., Pappas, G.J.: Robustness of temporal logic specifications for continuous-time signals. *Theoretical Computer Science* **410**(42), 4262–4291 (2009). <https://doi.org/10.1016/j.tcs.2009.06.021>
10. Haghighi, I., Jones, A., Kong, Z., Bartocci, E., Grosu, R., Belta, C.: Spatel: a novel spatial-temporal logic and its applications to networked systems. In: Girard, A., Sankaranarayanan, S. (eds.) *Proceedings of the 18th International Conference on Hybrid Systems: Computation and Control, HSCC'15, Seattle, WA, USA, April 14–16, 2015*. pp. 189–198. ACM (2015). <https://doi.org/10.1145/2728606.2728633>, <https://doi.org/10.1145/2728606.2728633>
11. Kapoor, P., Mizuta, K., Kang, E., Leung, K.: STL_{CG}++: A masking approach for differentiable signal temporal logic specification. *IEEE Robotics and Automation Letters* **10**(9), 9240–9247 (2025). <https://doi.org/10.1109/LRA.2025.3588389>
12. Leung, K., Aréchiga, N., Pavone, M.: Back-propagation through signal temporal logic specifications: Infusing logical structure into gradient-based methods. In: *Algorithmic Foundations of Robotics XIV (WAFR)*. Springer Proceedings in Advanced Robotics, vol. 17, pp. 432–449. Springer (2021). https://doi.org/10.1007/978-3-030-66723-8_26
13. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT)*. Lecture Notes in Computer Science, vol. 3253, pp. 152–166. Springer (2004). https://doi.org/10.1007/978-3-540-30206-3_12
14. Nenzi, L., Bartocci, E., Bortolussi, L., Silvetti, S., Loret, M.: MoonLight: A lightweight tool for monitoring spatio-temporal properties. *International Journal on Software Tools for Technology Transfer* **25**(4), 503–517 (2023). <https://doi.org/10.1007/s10009-023-00710-5>
15. Nickovic, D., Yamaguchi, T.: RTAMT: Online robustness monitors from STL. In: *Automated Technology for Verification and Analysis (ATVA)*. Lecture Notes in Computer Science, vol. 12302, pp. 564–571. Springer (2020). https://doi.org/10.1007/978-3-030-59152-6_34
16. Pant, Y.V., Abbas, H., Mangharam, R.: Smooth operator: Control using the smooth robustness of temporal logic. In: *IEEE Conference on Control Technology and Applications (CCTA)*. pp. 1235–1240. IEEE (2017). <https://doi.org/10.1109/CCTA.2017.8062628>
17. Raman, V., Donzé, A., Maasoumy, M., Murray, R.M., Sangiovanni-Vincentelli, A., Seshia, S.A.: Model predictive control with signal temporal logic specifications. In: *53rd IEEE Conference on Decision and Control (CDC)*. pp. 81–87. IEEE (2014). <https://doi.org/10.1109/CDC.2014.7039363>
18. Thomsen, A.K., Madsen, N.V.S., Evans, V.T., Wright, T.D., Esterle, L., Larsen, P.G.: mstlo: Efficient online monitoring of signal temporal logic. *arXiv preprint arXiv:2605.26847* (2026)
19. Vazquez-Chanlatte, M.: mvcisback/py-metric-temporal-logic: v0.1.1 (Jan 2019). <https://doi.org/10.5281/zenodo.2548862>, <https://doi.org/10.5281/zenodo.2548862>
20. Yamaguchi, T., Hoxha, B., Nickovic, D.: RTAMT – runtime robustness monitors with application to CPS and robotics. *International Journal on Software Tools for Technology Transfer* **26**(1), 79–99 (2024). <https://doi.org/10.1007/s10009-023-00720-3>
21. Zhang, Z., An, J., Arcaini, P., Hasuo, I.: Online causation monitoring of signal temporal logic. In: Enea, C., Lal, A. (eds.) *Computer Aided Verification - 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part I*. Lecture Notes in Computer Science, vol. 13964, pp. 62–84. Springer

- (2023). https://doi.org/10.1007/978-3-031-37706-8_4, https://doi.org/10.1007/978-3-031-37706-8_4
22. Zhang, Z., An, J., Arcaini, P., Hasuo, I.: Caumon: A tool for online monitoring against signal temporal logic. *Sci. Comput. Program.* **253**, 103499 (2026). <https://doi.org/10.1016/J.SCICO.2026.103499>, <https://doi.org/10.1016/j.scico.2026.103499>